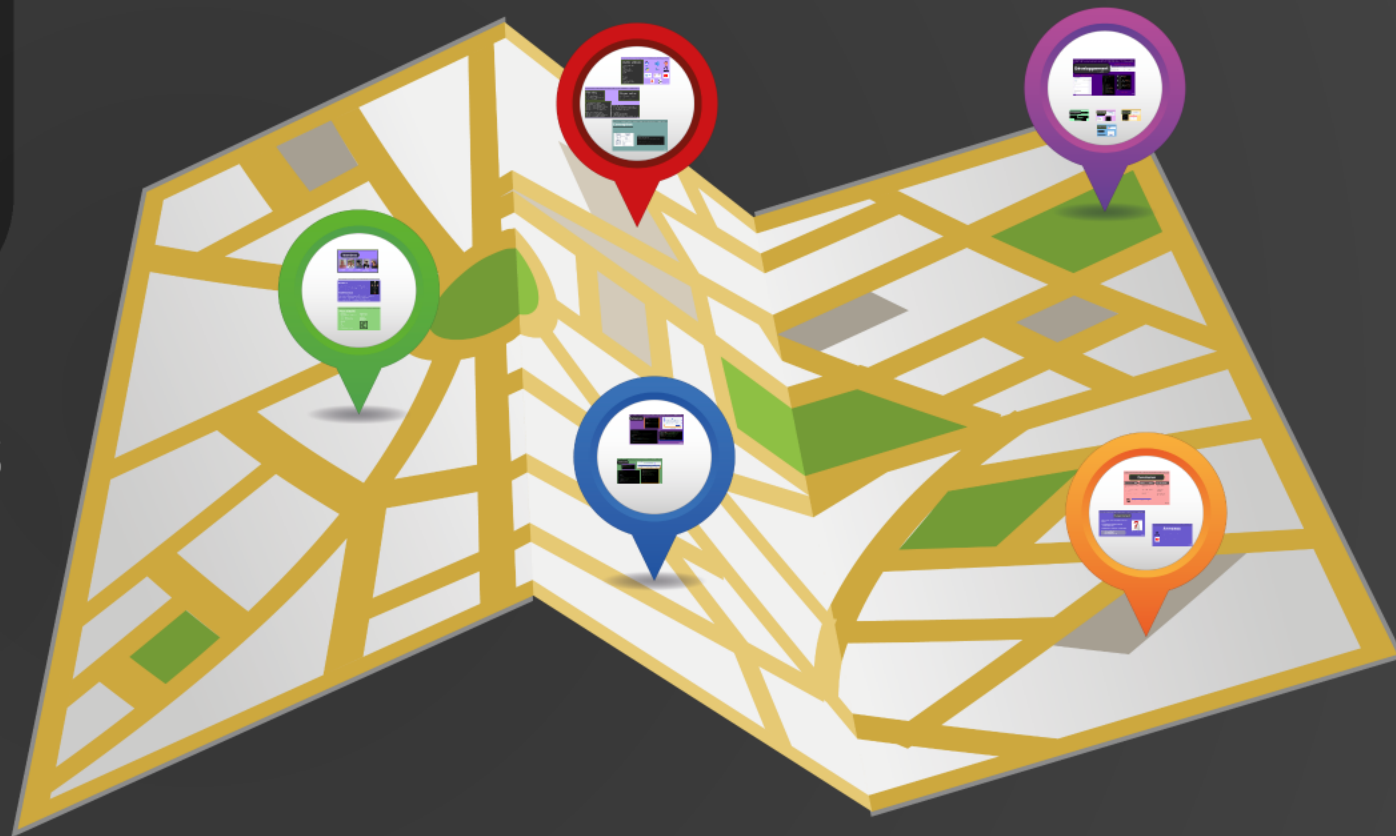


SAE 2.3

Mettre en place une solution informatique pour
l'entreprise

Grégoire JURY--VERMOT DES ROCHES
Bastien LORENZI
Florian PISTOLET



Membres



M. CANALDA
Professeur



M. DEDU
Professeur



Grégoire



Florian



Bastien

Jury

Groupe n°15 projet n°5

Contexte

Le projet consiste à créer une application qui affiche chaque jour le vent des lieux de résidence des étudiants de BUT1 R&T. Les données sont récupérées via fichier JSON et une API météo, puis affichées sur une interface web.



Problématiques

Comment recenser les résidences principale et secondaire de chaque étudiant ainsi que leurs groupes ?

Comment récupérer automatiquement les données de vent de chaque ville via une API météo ?

Comment stocker ces informations dans une base de données fiable et structurée ?

Comment afficher le vent du jour et son historique pour chaque étudiant ?

Comment calculer et visualiser le vent médian du groupe ?

Et enfin, comment rendre tout cela accessible via une application simple et efficace ?

Plan détaillé

1. Outils utilisés

2. Planning, Organisation, Développement

-Tests réalisés

- Tests manuels fonctionnels
- Tests d'import des données
- Tests de la clé API
- Cas limite

3.Conception

- MCD
- MPD

4.Développement

- Fichier Json
- Création de la base de donnée

5.Fonctionnalités

6.Vue Front accueil

- HTML, CSS

7.Conclusion

8.Remerciement

9.Annexes



Outils utilisés

- Visual Studio code
- Python
- SQLite (locale)
- Flask framework
- HTML
- CSS
- GitHub
- Youtube
- Prezi
- OpenWeatherMap
- Claude IA



Planning

Jour n°1 : Conception de la base de donnée (json)

Jour n°2 : Conception de l'application

Jour n°3 : Revue application / rapports et présentation

Jour n°4 : Rapports et présentation

Organisation

Grégoire : Création Diapositive / aide code

Florian : Création des rapports / aide code

Bastien : Développement du code

I. Tests manuels fonctionnels :

- Tests formulaire de recherche avec le nom
- Vérification de l'historique de recherche global
- Vérification de l'historique de recherche personnel
- Vérification de la fonctionnalité des boutons
- Vérification du calcul du ventilomètre collectif

II. Tests d'import des données :

- Lancement du script de création de la db
- Lancement du script d'importation du fichier JSON
- Vérification qu'aucun étudiant soit en double

III. Tests de la clé API

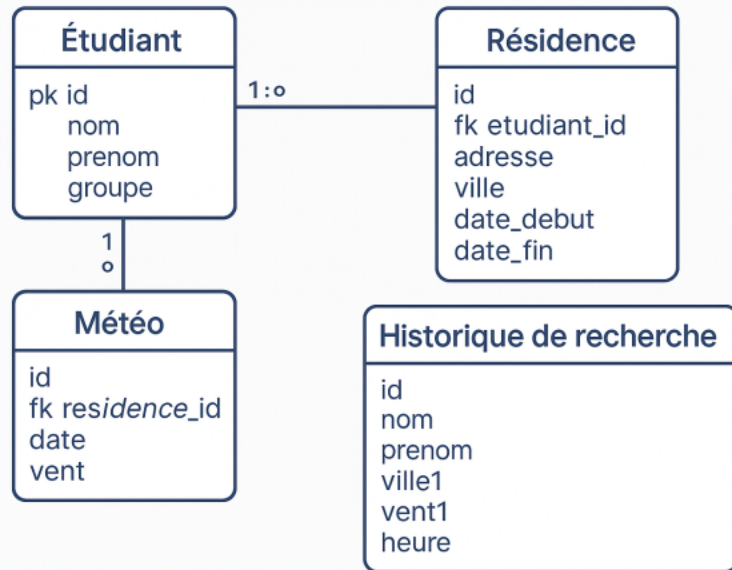
- Lancement du script amenant les données de la clé API
- Tests de la clé API avec villes existantes / non existantes
- Tests si les données récoltées s'affichent

IV. Autre cas

- Étudiant sans résidence secondaire
- Étudiant inexistant (mauvais nom)
- Ville inconnue de l'API météo (hors France / mal écrite)

Conception

MCD



MPD

```
Etudiant(id, nom, prenom, groupe)
```

```
Résidence(id, etudiant_id, adresse, ville, date_debut, date_fin)
```

```
Meteo(id, residence_id, date, vent)
```

```
HistoriqueRecherche(id, nom, prenom, ville1, vent1, ville2, vent2, heure)
```

Développement

Fichier JSON

Réponses au questionnaire

1	Horodateur	Nom & Prénom (ex : Lorenzi Bastien)	Ville de résidence prin	Ville de résidence secondaire (opti)
2	14/04/2025 09:48:41	RUFFINONI Noah	Petit-Noir	
3	14/04/2025 09:48:42	Errajifi Ayoub	Seloncourt	
4	14/04/2025 09:49:24	Minghi jules	Etupes	Bourges
5	14/04/2025 09:49:48	LORENZI Bastien	Lure	Montbéliard
6	14/04/2025 09:50:24	JVDR Grégoire	Ecot	

Questionnaire pour les étudiants

Lieu de résidence

Merci de renseigner votre lieu de résidence et éventuellement votre habitat secondaire

bastienlorenzi70@gmail.com

Changer de compte

Renvoyer le formulaire pour enregistrer les modifications

Non partagé

* Indique une question obligatoire

Nom & Prénom (ex : Lorenzi Bastien) *

Dedu Eugen

Ville de résidence principale *

Bavilliers

Ville de résidence secondaire (optionnel)

Votre réponse

Envoyer

```
392 {
393   "nom": "JURY--VERMOT DES ROCHES",
394   "prenom": "Grégoire",
395   "groupe": "LK1",
396   "residences": [
397     {
398       "adresse": "Inconnue 1",
399       "ville": "Écot",
400       "date_debut": "2024-01-01",
401       "date_fin": null
402     }
403   ],
404 },
405 {
406   "nom": "LORENZI",
407   "prenom": "Bastien",
408   "groupe": "LK1",
409   "residences": [
410     {
411       "adresse": "Inconnue 1",
412       "ville": "Montbéliard",
413       "date_debut": "2024-01-01",
414       "date_fin": null
415     },
416     {
417       "adresse": "Inconnue 2",
418       "ville": "Lure",
419       "date_debut": "2024-01-01",
420       "date_fin": "2024-02-04"
421     }
422   ],
423 }
```

```
614 {
615   "nom": "PISTOLET",
616   "prenom": "Florian",
617   "groupe": "LK2",
618   "residences": [
619     {
620       "adresse": "Inconnue 1",
621       "ville": "Montbéliard",
622       "date_debut": "2024-01-01",
623       "date_fin": null
624     },
625     {
626       "adresse": "Inconnue 2",
627       "ville": "Vesoul",
628       "date_debut": "2024-01-01",
629       "date_fin": "2024-02-04"
630     }
631   ],
632 }
```

Dossier .json

Développement

Création base de données

IV. Liaison des différentes tables

```
# === Importation des données depuis le fichier JSON ===
with open("etudiants.json", "r", encoding="utf-8") as f:
    data = json.load(f) # Chargement des données JSON en mémoire

# Parcourt chaque étudiant dans les données JSON
for etu in data:
    # Insertion de l'étudiant dans la table `etudiants`
    cursor.execute("INSERT INTO etudiants (nom, prenom, groupe) VALUES (?, ?, ?)",
                  | (etu["nom"], etu["prenom"], etu["groupe"]))
    etudiant_id = cursor.lastrowid # Récupère l'ID automatiquement généré

    # Insertion des résidences associées à cet étudiant
    for res in etu["residences"]:
        cursor.execute("""
            INSERT INTO residences (etudiant_id, adresse, ville, date_debut, date_fin)
            VALUES (?, ?, ?, ?, ?)
            """, (etudiant_id, res["adresse"], res["ville"], res["date_debut"], res["date_fin"]))
```

```
# Création de la table contenant les données météo associées à chaque résidence
cursor.execute("""
CREATE TABLE IF NOT EXISTS meteo (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    residence_id INTEGER NOT NULL,
    date TEXT NOT NULL,
    vent REAL,
    FOREIGN KEY (residence_id) REFERENCES residences(id)
)
""")
```

I. Supression de la base de données

```
# === Supprimer la base de données si elle existe déjà ===
if os.path.exists("students.db"):
    os.remove("students.db") # Supprime le fichier "students.db"
    print("Ancienne base supprimée.")
```

II. Création de la base de données students.db

```
# === Création d'une nouvelle base de données ===
conn = sqlite3.connect("students.db") # Crée un nouveau fichier de base de données
cursor = conn.cursor() # Initialise un curseur pour exécuter des requêtes SQL
```

III. Création des tables

```
# Création de la table des étudiants
cursor.execute("""
CREATE TABLE IF NOT EXISTS etudiants (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    prenom TEXT NOT NULL,
    groupe TEXT
)
""")
```

```
# Création de la table des résidences associées à chaque étudiant
cursor.execute("""
CREATE TABLE IF NOT EXISTS residences (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    etudiant_id INTEGER NOT NULL,
    adresse TEXT NOT NULL,
    ville TEXT NOT NULL,
    date_debut TEXT NOT NULL,
    date_fin TEXT,
    FOREIGN KEY (etudiant_id) REFERENCES etudiants(id)
)
""")
```

```
# Création de la table pour l'historique des recherches
cursor.execute("""
CREATE TABLE IF NOT EXISTS historique_recherches (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    ville1 TEXT,
    vent1 REAL,
    ville2 TEXT,
    vent2 REAL,
    heure TEXT
)
""")
```

Arrivé sur la page d'accueil

Fonctionnalités

```
# Récupération des vents du jour pour toutes les résidences, pour calculer le vent médian collectif
cursor.execute("SELECT vent FROM meteo WHERE date = date('now')")
vents = [row[0] for row in cursor.fetchall() if row[0] is not None]
if vents:
    vents.sort()
    n = len(vents)
    # Calcul de la médiane
    vent_collectif = vents[n // 2] if n % 2 == 1 else (vents[n // 2 - 1] + vents[n // 2]) / 2
```



Ventilomètre des Étudiants

 Ventilomètre collectif du jour : **5.08 m/s**

Rechercher

 Voir tous les étudiants

Rechercher

 Voir tous les étudiants

Résultat pour LORENZI Bastien

 Résidence principale : Montbéliard — Vent : 3.72 m/s

 Résidence secondaire : Lure — Vent : 2.57 m/s

 Historique personnel de LORENZI Bastien

2025-04-17 21:48:58 – LORENZI Bastien : Montbéliard (3.72 m/s) / Lure (2.57 m/s)

 Historique global

2025-04-17 21:48:58 – LORENZI Bastien : Montbéliard (3.72 m/s) / Lure (2.57 m/s)

 Vider l'historique

```
# Récupère les résidences associées à l'étudiant dont le nom est saisi
cursor.execute("""
SELECT e.id, e.nom, e.prenom, r.ville, r.date_fin, r.id
FROM etudiants e
JOIN residences r ON e.id = r.etudiant_id
WHERE e.nom = ?
""", (nom_recherche.upper(),))
rows = cursor.fetchall()

if rows:
    # Première résidence trouvée
    id, nom, prenom, ville1, _, res1_id = rows[0]
    ville2 = None # Ville secondaire
    res2_id = None # ID résidence secondaire

    # Recherche de la résidence secondaire (avec une date de fin non nulle)
    for row in rows[1:]:
        if row[4] is not None:
            ville2 = row[3]
            res2_id = row[5]

    # Récupère la vitesse du vent dans la résidence principale pour aujourd'hui
    cursor.execute("SELECT vent FROM meteo WHERE residence_id = ? AND date = date('now')", (res1_id,))
    vent1 = cursor.fetchone()
    vent1 = vent1[0] if vent1 else None

    # Récupère la vitesse du vent dans la résidence secondaire si elle existe
    vent2 = None
    if res2_id:
        cursor.execute("SELECT vent FROM meteo WHERE residence_id = ? AND date = date('now')", (res2_id,))
        res = cursor.fetchone()
        vent2 = res[0] if res else None

    # Insertion de la recherche dans l'historique
    cursor.execute("""
INSERT INTO historique_recherches (nom, prenom, ville1, vent1, ville2, vent2, heure)
VALUES (?, ?, ?, ?, ?, ?, ?)
""", (nom, prenom, ville1, vent1, ville2, vent2, now))
```

```

# Récupère les résidences associées à l'étudiant dont le nom est saisi
cursor.execute("""
    SELECT e.id, e.nom, e.prenom, r.ville, r.date_fin, r.id
    FROM etudiants e
    JOIN residences r ON e.id = r.etudiant_id
    WHERE e.nom = ?
""", (nom_recherche.upper(),))
rows = cursor.fetchall()

if rows:
    # Première résidence trouvée
    id_, nom, prenom, ville1, _, res1_id = rows[0]
    ville2 = None # Ville secondaire
    res2_id = None # ID résidence secondaire

    # Recherche de la résidence secondaire (avec une date de fin non nulle)
    for row in rows[1:]:
        if row[4] is not None:
            ville2 = row[3]
            res2_id = row[5]

    # Récupère la vitesse du vent dans la résidence principale pour aujourd'hui
    cursor.execute("SELECT vent FROM meteo WHERE residence_id = ? AND date = date('now')", (res1_id,))
    vent1 = cursor.fetchone()
    vent1 = vent1[0] if vent1 else None

    # Récupère la vitesse du vent dans la résidence secondaire si elle existe
    vent2 = None
    if res2_id:
        cursor.execute("SELECT vent FROM meteo WHERE residence_id = ? AND date = date('now')", (res2_id,))
        res = cursor.fetchone()
        vent2 = res[0] if res else None

    # Insertion de la recherche dans l'historique
    cursor.execute("""
        INSERT INTO historique_recherches (nom, prenom, ville1, vent1, ville2, vent2, heure)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (nom, prenom, ville1, vent1, ville2, vent2, now))

```

Fonctionnalités

```
@app.route("/etudiants") # Route pour afficher la liste des étudiants et leurs résidences
def etudiants():
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()

    # Récupère les informations des étudiants et de leurs résidences
    cursor.execute("""
        SELECT e.id, e.nom, e.prenom, e.groupe, r.ville, r.date_fin
        FROM etudiants e
        JOIN residences r ON e.id = r.etudiant_id
        ORDER BY e.id, r.date_fin IS NULL DESC, r.date_fin
    """)
    rows = cursor.fetchall()
    conn.close()

    etudiants_data = {} # Dictionnaire structurant les données pour chaque étudiant

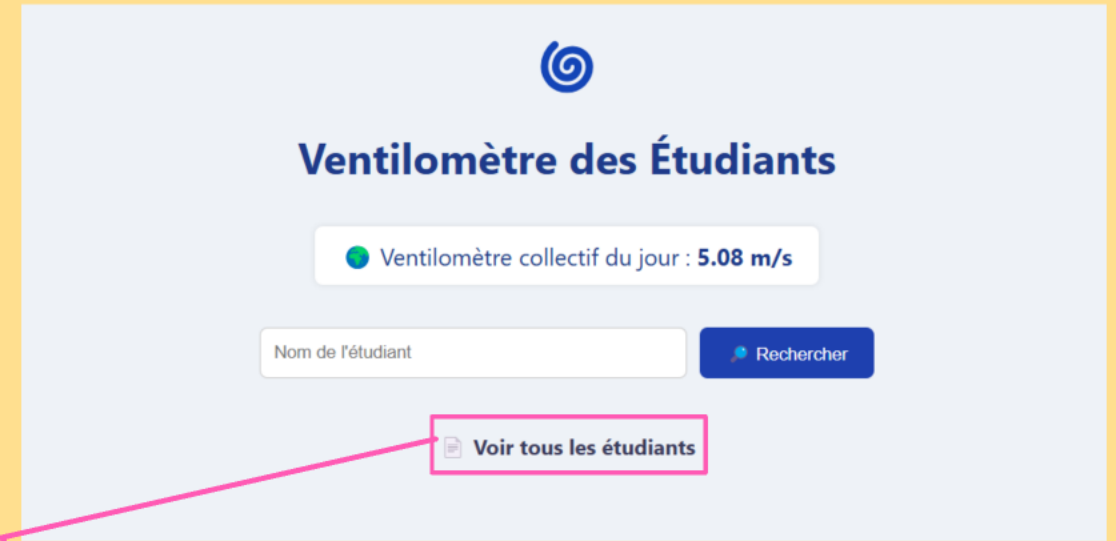
    for id_, nom, prenom, groupe, ville, date_fin in rows:
        # Si l'étudiant n'est pas encore enregistré dans le dictionnaire
        if id_ not in etudiants_data:
            etudiants_data[id_] = {
                "nom": nom,
                "prenom": prenom,
                "groupe": groupe,
                "ville_principale": None,
                "ville_secondaire": None
            }

        # Classe la résidence comme principale ou secondaire selon la date de fin
        if date_fin is None and etudiants_data[id_]["ville_principale"] is None:
            etudiants_data[id_]["ville_principale"] = ville
        elif date_fin is not None and etudiants_data[id_]["ville_secondaire"] is None:
            etudiants_data[id_]["ville_secondaire"] = ville

    # Conversion du dictionnaire en liste pour affichage
    etudiants = list(etudiants_data.values())

    # Rend la page HTML avec les données des étudiants
    return render_template("etudiants.html", etudiants=etudiants)
```

13/19



 **Liste des Étudiants** 

Groupe	Nom	Prénom	Ville principale	Ville secondaire
GB1	ARAGOU	Soulaym	Montbéliard	Paris
GB1	BOYER	Enzo	Montbéliard	Marseille
GB1	CHASSIGNOL	Ugo	Montbéliard	Foussemagne
GB1	DAVID	TOM	Morteau	Montbéliard
GB1	EL OMARI	Imrane	Montbéliard	Lille
GB1	ESSAJNANI	Farès	Montbéliard	Rennes
GB1	GALMICHE	Baptiste	Montbéliard	

12/19

```

@app.route("/etudiants") # Route pour afficher la liste des étudiants et leurs résidences
def etudiants():
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()

    # Récupère les informations des étudiants et de leurs résidences
    cursor.execute("""
        SELECT e.id, e.nom, e.prenom, e.groupe, r.ville, r.date_fin
        FROM etudiants e
        JOIN residences r ON e.id = r.etudiant_id
        ORDER BY e.id, r.date_fin IS NULL DESC, r.date_fin
    """)
    rows = cursor.fetchall()
    conn.close()

    etudiants_data = {} # Dictionnaire structurant les données pour chaque étudiant

    for id_, nom, prenom, groupe, ville, date_fin in rows:
        # Si l'étudiant n'est pas encore enregistré dans le dictionnaire
        if id_ not in etudiants_data:
            etudiants_data[id_] = {
                "nom": nom,
                "prenom": prenom,
                "groupe": groupe,
                "ville_principale": None,
                "ville_secondaire": None
            }

        # Classe la résidence comme principale ou secondaire selon la date de fin
        if date_fin is None and etudiants_data[id_]["ville_principale"] is None:
            etudiants_data[id_]["ville_principale"] = ville
        elif date_fin is not None and etudiants_data[id_]["ville_secondaire"] is None:
            etudiants_data[id_]["ville_secondaire"] = ville

    # Conversion du dictionnaire en liste pour affichage
    etudiants = list(etudiants_data.values())

    # Rend la page HTML avec les données des étudiants
    return render_template("etudiants.html", etudiants=etudiants)

```

13/19

Groupe	No
GB1	AR
GB1	BO
GB1	CH
GB1	DA
GB1	EL
GB1	ES
GB1	GA

Fonctionnalités

```
@app.route("/clear", methods=["POST"])
def clear_history():
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute("DELETE FROM historique_recherches")
    conn.commit()
    conn.close()
    return redirect(url_for("index"))
```

```
<a class="back-btn" href="{{ url_for('index') }}">← Retour à l'accueil</a>
```

Historique global

2025-04-17 21:48:58 – **LORENZI Bastien** : Montbéliard (3.72 m/s) / Lure (2.57 m/s)

 [Vider l'historique](#)

Thomas	Montbéliard
Semi	Montbéliard
Jad	Montbéliard
Tom	Morteau

[← Retour à l'accueil](#)

Vue Front accueil

```
input[type=text] {
  padding: 10px;
  width: 300px; /* Largeur fixe du champ de recherche */
  border-radius: 6px;
  border: 1px solid #ccc; /* Bordure grise claire */
}

button {
  padding: 10px 20px;
  background-color: #1e40af; /* Bleu vif pour se démarquer */
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer; /* Change le curseur en main au survol */
}

.students-link {
  margin-top: 1.5rem;
  text-align: center;
}
```



```
<body>
  <!-- Logo emoji affiché en haut -->
  <div class="logo">🌀</div>

  <!-- Titre principal de la page -->
  <h1> Ventilomètre des Étudiants </h1>

  <!-- Bloc conditionnel : affiché seulement si une valeur collective de vent est disponible -->
  {% if vent_collectif %}
  <div class="vent-median">
    🌍 Ventilomètre collectif du jour : <strong>{{ vent_collectif }} m/s</strong>
  </div>
  {% endif %}

  <!-- Formulaire de recherche d'un étudiant par son nom -->
  <form method="POST"> <!-- Les données sont envoyées au serveur en POST -->
    <input type="text" name="nom" placeholder="Nom de l'étudiant" required> <!-- Champ de saisie obligatoire -->
    <button type="submit">🔍 Rechercher</button> <!-- Bouton avec emoji loupe -->
  </form>

  <!-- Lien vers la page listant tous les étudiants -->
  <div class="students-link">
    <p>📄 <a href="{{ url_for('etudiants') }}">Voir tous les étudiants</a></p>
  </div>
```

```
body {
  font-family: 'Segoe UI', sans-serif; /* Police moderne et lisible */
  background-color: #eef2f7; /* Couleur de fond douce pour le confort visuel */
  margin: 0; /* Supprime la marge par défaut du navigateur */
  padding: 2rem; /* Ajoute de l'espace intérieur autour du contenu */
  display: flex;
  flex-direction: column; /* Organisation verticale du contenu */
  align-items: center; /* Centrage horizontal du contenu */
}

h1 {
  color: #1e3a8a; /* Titre en bleu foncé */
  text-align: center;
}
```

Vue Front étudiants

```
body {
  font-family: 'Segoe UI', sans-serif; /* Police moderne et lisible */
  background-color: #eef2f7; /* Fond bleu très clair */
  margin: 0; /* Aucun espace autour du body */
  padding: 2rem; /* Espace intérieur autour du contenu */
  display: flex; /* Organisation flexible du contenu */
  flex-direction: column; /* Organisation verticale */
  align-items: center; /* Centrage horizontal */
}

h1 {
  color: #1e3a8a; /* Bleu foncé pour le titre */
}
```

Liste des Étudiants

Groupe	Nom	Prénom	Ville principale	Ville secondaire
GB1	ARAGOU	Soulaym	Montbéliard	Paris
GB1	BOYER	Enzo	Montbéliard	Marseille

```
<!-- Titre principal de la page -->
<h1> 📖 Liste des Étudiants 📖 </h1>

<!-- Tableau affichant les données des étudiants -->
<table>
  <thead>
    <tr>
      <!-- En-têtes des colonnes du tableau -->
      <th>Groupe</th>
      <th>Nom</th>
      <th>Prénom</th>
      <th>Ville principale</th>
      <th>Ville secondaire</th>
    </tr>
  </thead>
  <tbody>
    <!-- Boucle Jinja : pour chaque étudiant dans la variable "etudiants", on crée une ligne -->
    {% for etu in etudiants %}
    <tr>
      <td>{{ etu.groupe }}</td> <!-- Affiche le groupe -->
      <td>{{ etu.nom }}</td> <!-- Affiche le nom -->
      <td>{{ etu.prenom }}</td> <!-- Affiche le prénom -->
      <td>{{ etu.ville_principale or "" }}</td> <!-- Affiche la ville principale, ou vide si non définie -->
      <td>{{ etu.ville_secondaire or "" }}</td> <!-- Affiche la ville secondaire, ou vide si non définie -->
    </tr>
    {% endfor %}
  </tbody>
</table>
```

```
table {
  width: 90%; /* Largeur relative à l'écran */
  max-width: 900px; /* Largeur maximale pour éviter un affichage trop large */
  border-collapse: collapse; /* Supprime les doubles bordures entre cellules */
  margin-top: 30px; /* Espace au-dessus du tableau */
  background: white; /* Fond blanc du tableau */
  box-shadow: 0 0 10px rgba(0,0,0,0.1); /* Ombre douce autour du tableau */
  border-radius: 8px; /* Coins arrondis */
  overflow: hidden; /* Empêche les débordements visuels */
}

th, td {
  padding: 12px 15px; /* Espacement intérieur des cellules */
  text-align: left; /* Alignement du texte à gauche */
}

th {
  background-color: #1e3a8a; /* Fond bleu foncé pour les en-têtes */
  color: white; /* Texte blanc */
}

tr:nth-child(even) {
  background-color: #f2f2f2; /* Alternance de couleur : gris clair */
}

tr:nth-child(odd) {
  background-color: #ffffff; /* Alternance de couleur : blanc */
}
```

Conclusion

Ce que nous avons réussi :

- Application web pour visualiser le vent des étudiants
- Backend Flask, base de données SQLite
- Maniement de JSON
- API météo pour le vent en temps réel

Ce que cela nous a apporté :

- Expérience concrète en développement
- Meilleure compréhension de la manipulation de données
- Travail collaboratif sur un projet complet

Améliorations possibles :

- Authentification des utilisateurs
- Affichage des tendances météo (graphiques)
- Une optimisation pour une interface mobile éventuellement



Mieux comprendre le fonctionnement ainsi que la documentation :

https://github.com/Sneaksi/SAE_2.3_ventilometre_G15c

Remerciement

- **À nos enseignants, pour leur accompagnement au cours du projet**
- **À l'IUT de Belfort-Montbéliard, pour les outils et l'environnement de travail**
- **À notre équipe, pour la collaboration et l'investissement**

**Contact : florian.pistolet@gmail.com
gregjvdr@gmail.com
bastienlorenzi70@gmail.com**



Avez vous des questions ?

Annexes



https://github.com/Sneaksi/SAE_2.3_ventilometre_G15c



<https://www.youtube.com/watch?v=1aZqyv6otUw>